TEMA 1: Arquitecturas y Tecnologías de Programación sobre Clientes Web

1.1. Caracterización y diferenciación de los modelos de ejecución de código en el servidor y en el cliente Web

- **Resultado de Aprendizaje Asociado: RA1** (*Selecciona las arquitecturas y tecnologías de programación sobre clientes Web, identificando y analizando las capacidades y características de cada una*).

- **Criterio Curricular Oficial: CE 1.a** (*Se han caracterizado y diferenciado los modelos de ejecución de código en el servidor y en el cliente Web*).

- **Ponderación del Criterio:** (**16,67% del RA1** | **0,833% sobre la calificación final del módulo**).

Cualquier aplicación web funciona mediante el modelo cliente/servidor. Este modelo reparte el trabajo entre dos equipos conectados a través de internet: el cliente (el ordenador, móvil o tableta de la persona que navega) y el servidor (uno o varios ordenadores remotos que guardan los datos y la lógica principal).

A. Contexto Histórico y Estandarización de la Web

- **Origen de la Web:** Nació en 1989 en el laboratorio europeo de física de partículas (CERN). Su creador, Tim Berners-Lee, buscaba un sistema sencillo para que los científicos pudieran compartir documentos con enlaces entre sí.

- **El consorcio W3C (World Wide Web Consortium):** Es el organismo internacional encargado de crear las reglas y estándares oficiales para que las páginas web funcionen igual en cualquier navegador del mundo. Su delegación en España se localiza en www.w3c.es.

- **La nube (cloud computing):** Antiguamente, las empresas mantenían sus propios ordenadores servidores en dependencias físicas locales. En la actualidad, la mayoría externaliza o contrata espacio y potencia de cálculo bajo demanda en plataformas de internet como AWS (Amazon Web Services).

- **Especialización del trabajo técnico:** Con el incremento de complejidad de las aplicaciones web, los profesionales se distribuyen en roles especializados:

- **Diseñadores (UX/UI):** Diseñan la paleta de colores, la tipografía y la distribución de la interfaz.

- **Programadores de cliente (Front-end):** Implementan los componentes interactivos, menús y pantallas con las que interactúa el usuario.

- **Programadores de servidor (Back-end):** Desarrollan los servicios para persistir datos, procesar transacciones seguras y proteger los accesos.

- **Administradores de bases de datos (DBA):** Gestionan, optimizan y garantizan la integridad de los datos almacenados.

B. El Entorno Servidor (Back-end)

El *back-end* constituye la infraestructura no visible de una aplicación web para el usuario final.

- **¿Dónde se ejecuta?** En servidores dedicados o instancias de computación en la nube. El usuario final nunca tiene acceso directo al código fuente allí implementado.

- **Tareas principales:**

  - Autenticar la identidad de los usuarios (credenciales y sesiones).

  - Realizar cobros y pagos bancarios de manera segura.

  - Conectarse a sistemas gestores de bases de datos para almacenar y consultar registros (pedidos, expedientes, inventarios):

    - *Bases de datos relacionales (SQL):* Almacenan información en tablas con filas y columnas (MySQL, MariaDB, PostgreSQL, Oracle).

    - *Bases de datos documentales (NoSQL):* Gestionan la información mediante bloques y documentos (MongoDB).

- **Tecnologías y lenguajes habituales en servidor:** PHP, Java, Python, Node.js (JavaScript del lado del servidor) o C# (.NET).

C. El Entorno Cliente (Front-end)

El *front-end* representa la capa de presentación que se dibuja directamente en la pantalla del dispositivo del cliente (ordenador, teléfono o tableta).

- **¿Dónde se ejecuta?** Localmente dentro del navegador web del usuario. El cliente descarga los archivos a través de la red y el navegador los interpreta utilizando los ciclos de CPU y la memoria RAM del propio dispositivo local.

- **Los tres pilares del front-end:**

    - **HTML:** El armazón estructural. Define los elementos del documento (títulos, párrafos, tablas, imágenes y formularios).

    - **CSS:** El diseño y presentación visual. Controla colores, tipografías, alineaciones y maquetación adaptativa (*responsive*).

    - **JavaScript:** La capa lógica interactiva. Permite responder a eventos de ratón/teclado, alterar dinámicamente el documento y validar información localmente de manera inmediata.

- **Premisa de seguridad (Visibilidad del código):** El código que corre en el cliente es completamente público. Cualquier usuario puede abrir las herramientas de desarrollador (**F12 / DevTools**) y ver, auditar o modificar el HTML y JavaScript en tiempo de ejecución. En consecuencia, las claves de cifrado, contraseñas y operaciones contables o de seguridad jamás deben ubicarse exclusivamente en el cliente.

D. Tabla Comparativa: Cliente vs. Servidor

Dimensión	Entorno Cliente (Front-end)	Entorno Servidor (Back-end)
Entorno de ejecución	Navegador del usuario (Chrome, Firefox, Safari, Edge)	Servidor remoto físico o instancia en la nube (AWS)
Visibilidad del código	**Público:** Accesible e inspeccionable con F12 / DevTools	**Privado:** Reside de forma estricta en el servidor
Tecnologías clave	HTML5, CSS3, JavaScript (React, Vue, Angular)	PHP, Python, Java, Node.js, C# (.NET)

Acceso a datos	**Indirecto:** Peticiones HTTP a través de internet	**Directo:** Conexión nativa a motores SQL o NoSQL
Consumo de recursos	CPU, memoria RAM y batería del dispositivo local	Potencia de cómputo y memoria del servidor
Latencia de respuesta	Inmediata para acciones visuales locales	Sujeta a la latencia de red y carga del servidor
Nivel de seguridad	**Bajo:** Modificable por el cliente mediante scripts	**Alto:** Núcleo fiable para cobros, permisos y roles

E. Evolución de la Navegación: De la Web Tradicional a la Web Moderna

- **1. El modelo clásico tradicional:**
    - Cada clic en un enlace o envío de un formulario disparaba una petición síncrona al servidor.
    - El servidor procesaba la solicitud completa y ensamblaba un nuevo documento HTML íntegro.
    - La interfaz del usuario sufría una pantalla en blanco y un parpadeo visible al recargar toda la estructura desde cero.
- **2. El modelo moderno con JavaScript (Single Page Applications - SPA):**
    - La aplicación descarga la plantilla y recursos base una única vez al iniciar.
    - Cuando el usuario interactúa, aplica filtros o cambia de sección, la página no se recarga completamente.
    - JavaScript solicita de forma asíncrona únicamente los datos necesarios empaquetados en **JSON** y actualiza selectivamente las partes del árbol visual que cambian.

F. Criterios de Asignación de Tareas: Cliente vs. Servidor

Para decidir la ubicación de un procedimiento se aplica el principio: **lo que aporta usabilidad e inmediatez va en el cliente; lo que requiere integridad y seguridad va en el servidor**.

- **Operaciones propias del Cliente (*Front-end*):**
    - Desplegar y ocultar menús, ventanas modales o acordeones visuales.
    - Modificar estilos y colores en respuesta a eventos del ratón (*hover*, clics).
    - Ordenar y filtrar colecciones de datos ya cargadas en memoria.
- **Operaciones exclusivas del Servidor (*Back-end*):**
    - Realizar cobros con pasarelas de pago bancarias (el importe final se calcula en servidor para evitar alteraciones maliciosas en el navegador).
    - Ejecutar consultas complejas sobre miles o millones de registros en bases de datos.
    - Comprobar privilegios y roles de usuario antes de conceder accesos administrativos.

G. Vocabulario Técnico

- **Cliente:** Dispositivo y software navegador utilizado por el usuario para interactuar con la aplicación web.
- **Servidor:** Máquina conectada permanentemente a la red que aloja los servicios, gestiona las reglas de negocio y responde a las solicitudes de los clientes.
- **Renderizar:** Proceso computacional mediante el cual el motor del navegador interpreta el código HTML, CSS y JavaScript para pintar los elementos visuales en pantalla.
- **Base de datos:** Sistema informático orientado al almacenamiento estructurado, persistencia y consulta ágil de grandes volúmenes de datos.
- **DevTools:** Conjunto de utilidades de diagnóstico integradas en los navegadores (accesibles con F12) que permiten auditar el DOM, monitorizar peticiones de red y depurar código JavaScript.

Actividad Práctica Guiada: Tiempos de ejecución en el Navegador

Código Base

Archivo catalogo.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Práctica: Consumo de CPU/RAM en Cliente</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 30px;
      line-height: 1.5;
    }
    .panel {
      max-width: 600px;
      padding: 20px;
      border: 1px solid #ccc;
      border-radius: 6px;
      background-color: #f9f9f9;
    }
    button {
      padding: 10px 15px;
      margin-right: 10px;
      margin-bottom: 10px;
      cursor: pointer;
      font-weight: bold;
      background-color: #0056b3;
      color: white;
      border: none;
      border-radius: 4px;
    }
    button:hover {
      background-color: #003d80;
    }
    #consolaVisual {
      margin-top: 15px;
      padding: 12px;
      background: #222;
      color: #00ff66;
      font-family: 'Courier New', Courier, monospace;
      font-size: 13px;
```

```
      border-radius: 4px;
      min-height: 80px;
      white-space: pre-line;
    }
  </style>
</head>
<body>

  <div class="panel">
    <h2>Auditoría de Rendimiento en el Cliente (Front-end)</h2>
    <p>Pulsa los botones y abre las <strong>Herramientas de Desarrollador (F12)</strong> para
ver las etiquetas temporales de la CPU del navegador.</p>

    <button id="btnGenerar">1. Generar 150.000 Productos</button>
    <button id="btnOrdenar">2. Ordenar por Precio</button>
    <button id="btnFiltrar">3. Filtrar Informática</button>

    <div id="consolaVisual">Esperando interacción del usuario...</div>
  </div>

  <script>// Variable global en el ámbito de cliente para retener la carga en memoria RAM
let catalogoProductos = [];

const categoriasDisponibles = ["Informática", "Telefonía", "Hogar", "Audio", "Oficina"];
const salida = document.getElementById("consolaVisual");

// 1. GENERACIÓN DE DATOS (Carga en memoria RAM del dispositivo cliente)
document.getElementById("btnGenerar").addEventListener("click", () => {
  salida.textContent = "Generando 150.000 registros en la RAM del cliente...";

  // Iniciamos el cronómetro de la consola
  console.time("Tiempo de generación en RAM");

  catalogoProductos = [];
  const TOTAL_REGISTROS = 150000;

  for (let i = 1; i <= TOTAL_REGISTROS; i++) {
    catalogoProductos.push({
      id: i,
      nombre: `Producto ${i}`,
      precio: parseFloat((Math.random() * 900 + 10).toFixed(2)),
      categoria: categoriasDisponibles[Math.floor(Math.random() * categoriasDisponibles.length)]
    });
  }
```

```
  // Detenemos el cronómetro
  console.timeEnd("Tiempo de generación en RAM");

  salida.textContent = `Generados ${TOTAL_REGISTROS.toLocaleString()} productos en
memoria RAM local.\nRevisa la consola (F12) para ver el tiempo exacto.`;
});

// 2. ORDENACIÓN EN CLIENTE (Consumo intensivo de CPU local)
document.getElementById("btnOrdenar").addEventListener("click", () => {
  if (catalogoProductos.length === 0) {
    salida.textContent = "Primero genera los productos.";
    return;
  }

  salida.textContent = "Ordenando catálogo por precio mediante la CPU del navegador...";

  // Medición con timer
  console.time("Tiempo de ordenación (CPU Cliente)");

  // Algoritmo de ordenación con predicado
  catalogoProductos.sort((a, b) => a.precio - b.precio);

  console.timeEnd("Tiempo de ordenación (CPU Cliente)");

  salida.textContent = `Ordenación completada.\n- Producto más barato:
${catalogoProductos[0].precio} €\n- Producto más caro:
${catalogoProductos[catalogoProductos.length - 1].precio} €\n(Consumo de cómputo
transferido con éxito del servidor al cliente).`;
});

// 3. FILTRADO DECLARATIVO (Procesamiento y asignación de nueva memoria)
document.getElementById("btnFiltrar").addEventListener("click", () => {
  if (catalogoProductos.length === 0) {
    salida.textContent = "Primero genera los productos.";
    return;
  }

  salida.textContent = "Filtrando productos de la categoría 'Informática'...";

  console.time("Tiempo de filtrado (.filter)");

  const productosFiltrados = catalogoProductos.filter(prod => prod.categoria === "Informática");
```

```
  console.timeEnd("Tiempo de filtrado (.filter)");

  salida.textContent = `Filtrado completado con éxito.\n- Registros encontrados:
${productosFiltrados.length.toLocaleString()}\nEl servidor no ha tenido que ejecutar ninguna
consulta SQL ni consumir hilos de procesamiento.`;
});
</script>
</body>
</html>
```

Modificaciones Guiadas que Debe Realizar el Alumnado

- **Modificación 1 (Invertir el criterio de ordenación):**
  - Localiza la función del método .sort() en script.js.
  - Modifícala para que ordene los artículos de **mayor a menor precio** (orden descendente).
  - Ajusta el mensaje de salida en visor para que muestre primero el artículo más caro y luego el más barato.
- **Modificación 2 (Cambiar la categoría del filtro):**
  - Modifica el predicado del método .filter() para que seleccione únicamente los productos de la categoría "Telefonía" en lugar de "Informática".
- **Modificación 3 (Aumento de volumen y prueba de estrés):**
  - Localiza la variable constante TOTAL y sustituye el valor de 150000 por 500000 registros.
  - Vuelve a pulsar los botones en orden y anota los nuevos tiempos mostrados en la consola.

Cuestionario de Análisis Técnico

Cuestión 1: Auditoría de Tiempos con console.time

- Anota el tiempo en milisegundos indicado por console.timeEnd("Tiempo de Ordenación") al procesar 150.000 elementos frente a los 500.000 elementos. ¿El tiempo consumido por el navegador crece de forma lineal o en una proporción distinta?

Cuestión 2: Ahorro de Procesamiento en el Servidor

- Si una tienda online tiene 10.000 usuarios consultando y reordenando el catálogo al mismo tiempo, ¿calcula el tiempo de ejecución total empleado por los navegadores de todos los 10.000 usuarios?, ¿qué beneficio supone para la empresa que este algoritmo se ejecute en el navegador de cada usuario en lugar de lanzar consultas ORDER BY continuas a la base de datos del servidor?

Cuestión 4: Límite Arquitectónico y Necesidad de Paginación

- Si el inventario contara con **8 millones de registros**, ¿sería viable descargarlos todos en un solo array en el navegador para que el cliente los ordene? Qué solución propondrías.